МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

ЗВІТ З

ЛАБОЛАТОРНОЇ №2

на тему **" Функції вищого порядку, lambda, композиція "**

Варіант 15

Виконала:

Студентка 2 курсу

групи ФЕП-22

Сулятицька Христина

Перевірив:

Аспірант

Остріков Данило Андрійович

Львів-2026

Мета роботи

Ознайомитися з функціями вищого порядку, lambda-виразами та композицією функцій; навчитися створювати функціональні конвеєри обробки даних; закріпити використання генераторних виразів і functools.reduce; реалізувати обробку записів без зміни початкових даних.

Теоретичні відомості

Функції вищого порядку — це функції, які приймають інші функції як аргументи або повертають функцію як результат. Такий підхід дає змогу передавати правила обробки окремо від основного алгоритму.

Lambda-вираз — це коротка анонімна функція, яку використовують для опису невеликої операції без створення окремої функції через def.

Генераторний вираз формує елементи послідовно, коли вони потрібні. Це дає змогу обробляти послідовності без створення додаткового списку в пам'яті.

Композиція функцій — спосіб об'єднання декількох функцій в одну. Результат однієї функції передається наступній, завдяки чому можна побудувати послідовний конвеєр перетворень.

Reduce — операція згортання послідовності до одного накопиченого результату. У Python вона реалізована в модулі functools і може використовуватися для підрахунку кількості записів та сумарних показників.

Хід роботи

Для 15-го варіанта розроблено програму аналізу результатів лабораторних робіт студентів. Для кожного запису зберігаються ідентифікатор, ім'я, група та список балів. Програма обчислює суму й середній бал, додає бонус студентам заданої групи, відбирає записи за мінімальним середнім балом і формує рейтинг Top-N.

1. Створення структури проєкту

Створено структуру проєкту лабораторної роботи:

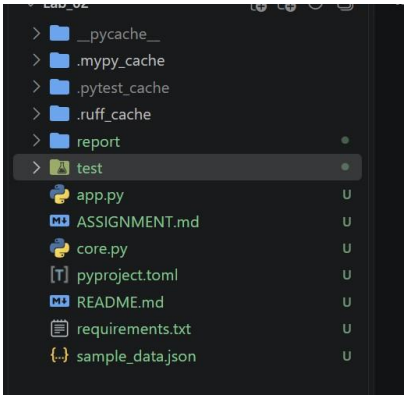


Рисунок 1 - Структура проєкту

2. Реалізація ядра програми у файлі core.py

У файлі core.py реалізовано типізовану структуру Student та функції для:

- обчислення суми та середнього результату лабораторних робіт;
- нарахування бонусу студентам заданої групи;
- відбору студентів за мінімальним середнім балом;
- нормалізації імен студентів;
- сортування результатів і формування рейтингу Top-N; □ обчислення загальної статистики за допомогою reduce.

Для опису запису використано TypedDict. Обов'язковими полями є id, name, group та lab_scores. Поля total_score, avg_score і final_score додаються під час обробки, тому позначені як NotRequired.

Функція with_score_statistics() проходить по записах студентів, обчислює суму балів за всі лабораторні роботи та середнє арифметичне. Для створення результату використовується розпакування словника **student, тому початковий запис не змінюється.



```python
from __future__ import annotations

from functools import reduce
from itertools import islice
from typing import Any, Callable, Iterable, Iterator, TypedDict, cast

from typing_extensions import NotRequired


class Student(TypedDict):

    id: int
    name: str
    group: str
    lab_scores: list[float]
    total_score: NotRequired[float]
    avg_score: NotRequired[float]
    final_score: NotRequired[float]


Stats = dict[str, float]
Transform = Callable[[Iterable[Student]], Iterable[Student]]


def compose(*functions: Callable[[Any], Any]) -> Callable[[Any], Any]:

    def composed(value: Any) -> Any:
        result = value
        for function in reversed(functions):
            result = function(result)
        return result

    return composed


def with_score_statistics() -> Transform:

    def transform(students: Iterable[Student]) -> Iterator[Student]:
        for student in students:
            scores = student["lab_scores"]
            total = float(sum(scores))
            average = total / len(scores) if scores else 0.0
            yield {
                **student,
                "total_score": total,
                "avg_score": average,
                "final_score": average,
            }

    return transform


def apply_group_bonus(group: str, bonus: float) -> Transform:

    def transform(students: Iterable[Student]) -> Iterator[Student]:
        for student in students:
            current = float(student.get("avg_score", 0.0))
            final = (
                min(100.0, current + bonus) if student["group"] == group else current
            )
            yield {**student, "final_score": final}

    return transform


def only_admitted(min_average: float) -> Transform:

    return lambda students: (
        student
        for student in students
        if float(student.get("avg_score", 0.0)) >= min_average
    )


def normalize_names() -> Transform:
    return lambda students: (
        {**student, "name": student["name"].strip().title()} for student in students
    )


def sort_and_take(top_n: int) -> Transform:
    def transform(students: Iterable[Student]) -> list[Student]:
        ordered = sorted(
            students,
            key=lambda student: (
                -float(student.get("final_score", 0.0)),
                student["id"],
            ),
        )
        return list(islice(ordered, max(0, top_n)))

    return transform


def build_pipeline(
    top_n: int = 5,
    bonus_group: str = "ІП3-21",
    bonus: float = 5.0,
    min_average: float = 60.0,
) -> Callable[[Iterable[Student]], list[Student]]:
    pipeline = compose(
        sort_and_take(top_n),
        only_admitted(min_average),
        apply_group_bonus(bonus_group, bonus),
        with_score_statistics(),
        normalize_names(),
    )
    return cast(Callable[[Iterable[Student]], list[Student]], pipeline)


def reduce_stats(students: Iterable[Student]) -> Stats:
    initial: Stats = {"count": 0.0, "total_score": 0.0, "avg_score_sum": 0.0}
    accumulated = reduce(
        lambda acc, student: {
            "count": acc["count"] + 1.0,
            "total_score": acc["total_score"] + float(student.get("total_score", 0.0)),
            "avg_score_sum": acc["avg_score_sum"]
            + float(student.get("avg_score", 0.0)),
        },
```

Рисунок 2 - core.py

Нарахування бонусу та фільтрація. Функція apply_group_bonus() додає задану кількість балів лише студентам вибраної групи. Фінальний бал не може перевищувати 100. Функція only_admitted() залишає записи, середній бал яких не нижчий за встановлений поріг. Для коротких перетворень використано lambda-вирази та генераторні вирази.

3. Реалізація композиції функцій та конвеєра обробки

Для формування рейтингу використано функцію sort_and_take(). Вона сортує студентів за фінальним балом у порядку спадання. Якщо значення однакові, додатковим ключем сортування є ID за зростанням. Після сортування функція повертає лише задану кількість записів. У build_pipeline() за допомогою compose() об'єднано всі етапи обробки. Композиція виконує функції справа наліво: спочатку нормалізуються імена, далі обчислюються сума та середній бал, застосовується бонус, виконується фільтрація і, нарешті, формується Top-N.

4. Реалізація згортання та агрегації

Функція reduce_stats() використовує functools.reduce для накопичення трьох показників: кількості студентів, суми балів лабораторних робіт та суми середніх балів. Після завершення згортання обчислюється середній бал вибірки. У програмі статистика формується для вже відібраного рейтингу Top-N.

5. Реалізація оболонки програми у файлі app.py

У файлі app.py реалізовано завантаження даних із JSON-файлу, перевірку структури записів, отримання параметрів командного рядка та виведення результатів. Для зчитування використовується функція load_students(), яка перевіряє, чи JSON містить список об'єктів і чи кожен запис має обов'язкові поля id, name, group та lab_scores.

Після завантаження даних створюється конвеєр build_pipeline() з файлу core.py. Параметрами запуску можна задати шлях до JSON-файлу, кількість студентів у рейтингу, групу для бонусу, розмір бонусу та мінімальний середній бал. Таким чином, правила обробки можна змінювати без редагування обчислювального ядра.



```python
"""Командний інтерфейс для демонстрації лабораторної роботи."""

from __future__ import annotations

import argparse
import json
from pathlib import Path
from typing import cast

from core import Student, build_pipeline, reduce_stats


def load_students(path: Path) -> list[Student]:
    """Зчитує записи студентів із JSON-файлу."""
    with path.open("r", encoding="utf-8") as source:
        data = json.load(source)

    if not isinstance(data, list):
        raise ValueError("JSON повинен містити список студентів.")

    students: list[Student] = []
    for item in data:
        if not isinstance(item, dict):
            raise ValueError("Кожен запис має бути JSON-об'єктом.")
        required = {"id", "name", "group", "lab_scores"}
        if not required.issubset(item):
            raise ValueError(f"У записі відсутні поля: {required - set(item)}")
        if not isinstance(item["lab_scores"], list):
            raise ValueError("Поле lab_scores повинно бути списком.")
        students.append(cast(Student, item))
    return students


def main() -> None:
    parser = argparse.ArgumentParser(
        description="Аналіз результатів лабораторних робіт студентів."
    )
    parser.add_argument("--data", type=Path, default=Path("sample_data.json"))
    parser.add_argument(
        "--top", type=int, default=5, help="Кількість записів у рейтингу"
    )
    parser.add_argument("--bonus-group", default="ІП3-21")
    parser.add_argument("--bonus", type=float, default=5.0)
    parser.add_argument("--min-average", type=float, default=60.0)
    args = parser.parse_args()

    students = load_students(args.data)
    pipeline = build_pipeline(
        top_n=args.top,
        bonus_group=args.bonus_group,
        bonus=args.bonus,
        min_average=args.min_average,
    )
    result = pipeline(students)

    print("РЕЙТИНГ СТУДЕНТІВ")
    print("-" * 78)
    print(
        f"{'Місце':<7}{'ID':<6}{'Ім'я':<24}{'Група':<14}{'Сума':>10}{'Середній':>10}{'Фінал':>9}"
    )
    print("-" * 78)
    for place, student in enumerate(result, start=1):
        print(
            f"{place:<7}{student['id']:<6}{student['name']:<24}"
            f"{student['group']:<14}{student['total_score']:>10.2f}"
            f"{student['avg_score']:>10.2f}{student['final_score']:>9.2f}"
        )

    stats = reduce_stats(result)
    print("\nЗАГАЛЬНА СТАТИСТИКА ДЛЯ СФОРМОВАНОГО TOP-N")
    print(f"Кількість студентів: {int(stats['count'])}")
    print(f"Сума балів лабораторних: {stats['total_score']:.2f}")
    print(f"Середній бал: {stats['avg_score']:.2f}")


if __name__ == "__main__":
    main()
```

Рисунок 3 - app.py

Для відображення результату програма виводить таблицю рейтингу з місцем студента, його ID, ім'ям, групою, сумою балів, середнім і фінальним балом.

Після цього викликається reduce_stats(), яка формує загальну статистику для сформованого рейтингу.

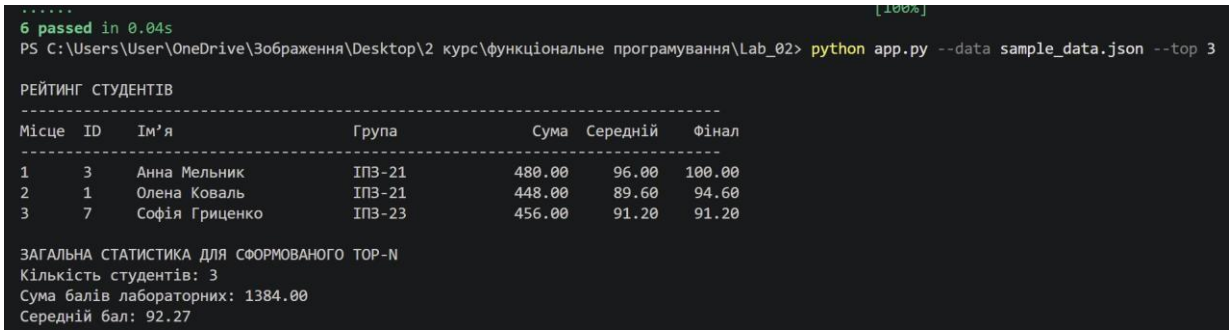


Рисунок 4 - Виведення рейтингу та загальної статистики

6. Створення та виконання тестів

Створено тести pytest для перевірки:

- правильності формування Top-N та сортування за двома ключами;
- відсутності зміни початкових записів студентів;
- нарахування бонусу лише вибраній групі;
- правильності виконання операції агрегації за допомогою reduce;
- порядку виконання функцій у композиції;

- роботи повного конвеєра з фільтрацією та обмеженням кількості результатів. Виконано тестування програми за допомогою команди python -m pytest -q. Усі шість тестів завершилися успішно. Додатково виконано перевірку типів за допомогою mypy, перевірку форматування Black та статичний аналіз Ruff. Усі перевірки пройшли без помилок.



Рисунок 5 - Тести для перевірки сортування, незмінності даних та бонусу

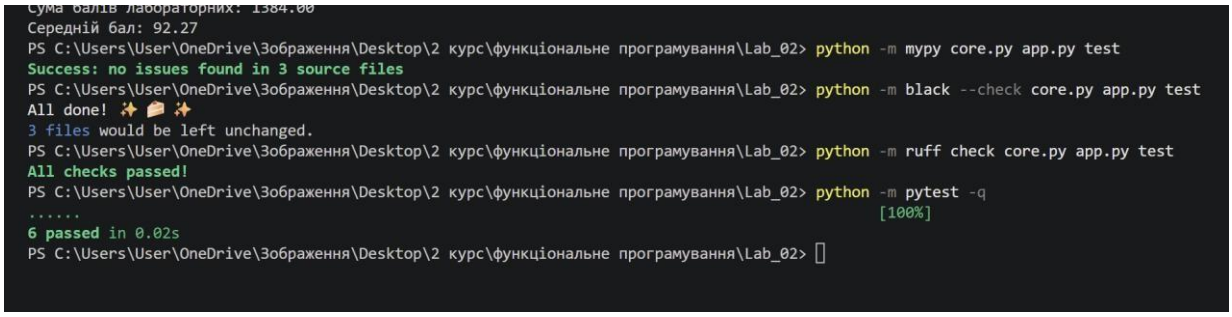


Рисунок 10 — Результати перевірок туру, Black, Ruff та pytest

Висновок

У ході лабораторної роботи було реалізовано програму для аналізу результатів лабораторних робіт студентів відповідно до 15-го варіанта. Програма зчитує дані з JSON-файлу, обчислює суму та середній бал лабораторних робіт, застосовує бонус для заданої групи, відбирає студентів за мінімальним середнім балом і формує рейтинг Top-N.

Під час розробки було використано функції вищого порядку, lambda-вирази, генераторні вирази та композицію функцій. Для об'єднання етапів обробки створено конвеєр build_pipeline(). Початкові записи не змінюються: під час

кожного перетворення створюються нові словники. Для обчислення загальних показників використано functools.reduce.

Також було реалізовано окрему оболонку програми у файлі app.py та створено автоматизовані тести за допомогою pytest. Усі шість тестів пройшли успішно. Додатково підтверджено коректність типів за допомогою mypy, форматування коду через Black та виконано статичну перевірку Ruff.

Таким чином, поставлену мету лабораторної роботи досягнуто.